

TripWiz — Folder 03: How the Parts Connect

Team: TripWiz — Ariel University · David Kitinberg, Amit Bitton, Sagi Hassid

Users

- **Users → Authentication:** sign up and sign in through Amazon Cognito (email + password, 6-digit verification code).
- **Users → Frontend:** open the React app in the browser — hosted on **AWS Amplify** in production (`https://main.d1qe3fi102rb8e.amplifyapp.com`); local dev via Vite (`npm run dev`).
- **Users → Notifications:** receive emails sent by Amazon SES (weather alerts, trip reminders, collaborator invites).
- **Admins → Admin portal:** sign in at `/admin-login` with a Cognito account in the **Admins** group (or a bootstrap-whitelisted email); manage users, trips, and trending content through the same React app.

Frontend ↔ Backend

- **Frontend → Authentication:** the app calls Cognito directly (SRP sign-in); no API Gateway on auth.
- **Frontend → API Gateway REST (HTTPS):** all trip, admin, place-search, route, attachment, settings, and validation calls via `frontend/src/services/api.js` .
- **Frontend → API Gateway WebSocket (WSS):** real-time collaboration on a trip (`join` , `leave` , `edit` , `cursor` , `ping`).
- **Frontend → S3 (presigned URLs):** trip attachment uploads and downloads use short-lived URLs issued by the REST API; files land in the private `TripWizDocumentsBucket` .
- **Frontend → Leaflet (client-side):** maps and stop markers render in the browser; road geometry comes from the backend route-calculation API (Amazon Location Service).

Authentication ↔ APIs

- **Cognito → REST API Gateway:** JWT ID token from sign-in is sent on REST requests; the built-in Cognito authorizer validates it before Lambda runs.
- **Cognito → WebSocket Authorizer:** access token is passed as `?token=` on `$connect` ; the `TripWiz-WebSocketAuthorizer` Lambda checks it before the connection is accepted.
- **Cognito → PostConfirmation Lambda:** after email verification, Cognito invokes `TripWiz-PostConfirmation` to seed default user preferences in DynamoDB and register the user's email as an SES identity (so trip emails can be delivered once verified).
- **Cognito (COGNITO_DEFAULT):** account verification codes are sent by Cognito's built-in email — not through the app's SES sender.

API Gateway ↔ Lambdas

- **REST API Gateway → TripWiz-Trips Lambda:** every HTTP route — trips CRUD, collaborators, attachments, on-demand validate, alerts/weather/fallbacks, route optimize/calculate, place search, user prefs, and all `/admin/*` endpoints.
- **REST (on-demand validate) → TripWiz-Validate:** `POST /trips/{tripId}/validate` on `TripWiz-Trips` asynchronously invokes `TripWiz-Validate` for immediate weather checks.
- **WebSocket API Gateway → TripWiz-WebSocketHandler Lambda:** `connect` , `disconnect` , `join` , `leave` , `edit` , `cursor` , and `ping` messages.
- **WebSocket API Gateway → TripWiz-WebSocketAuthorizer Lambda:** only on `$connect` .

Lambdas ↔ Data & Storage

- **TripWiz-Trips, TripWiz-WebSocketHandler, TripWiz-Validate, TripWiz-TripReminder, TripWiz-PostConfirmation** → **DynamoDB (TripWizTable)**: trips, stops, access records, alerts, WebSocket connection IDs, user preferences, and platform settings.
- **TripWiz-Trips** → **S3 (TripWizDocumentsBucket)**: presigned upload/download/delete for trip attachments (PDF, images, DOCX).
- **TripWiz-Trips** → **Secrets Manager**: optional mapping API key (TripWiz/MappingApiKey) at runtime.

Lambdas ↔ External services

- **TripWiz-Trips** → **Amazon Bedrock (Claude Haiku 4.5)**: AI itinerary reordering (POST /trips/{tripId}/optimize).
- **TripWiz-Trips** → **Amazon Location Service**: place autocomplete/search (TripWizPlaceIndex) and road-route calculation (TripWizRouteCalculator , Esri data).
- **TripWiz-Validate** → **Open-Meteo**: weather forecasts for each trip stop (free API, no key required).
- **TripWiz-Validate** → **Amazon Location Service**: nearby indoor fallback venues when weather conflicts are detected.

Scheduling ↔ Lambdas

- **EventBridge** → **TripWiz-Validate**: nightly scheduled weather validation for all upcoming trips (rate(1 day)).
- **EventBridge** → **TripWiz-TripReminder**: daily reminder job at 08:00 UTC (cron(0 8 * * ? *)).

Notifications chain

- **TripWiz-Validate** → **SNS (TripWiz-Alerts)**: publishes structured weather-alert payloads when conflicts are found.
- **SNS** → **TripWiz-SnsToSes Lambda**: alert payload delivered as an SNS subscription event.
- **TripWiz-SnsToSes** → **SES**: formatted HTML weather-alert emails to affected users.
- **TripWiz-TripReminder** → **SES**: upcoming-trip reminder emails (direct, no SNS).
- **TripWiz-Trips** → **SES**: collaborator-invite emails when a trip owner adds a co-planner.

Monitoring & deployment

- **All Lambdas** → **CloudWatch**: logs and metrics.
- **AWS SAM / CloudFormation**: backend/infra/template.yaml deploys and wires the entire backend stack (sam build + sam deploy); not a runtime data path.
- **AWS Amplify**: frontend/amplify.yaml builds and hosts the React frontend on every push to the connected branch.